Git Lab Practice 2 — Report

Oct 13, 2026 · @Lina Lassri

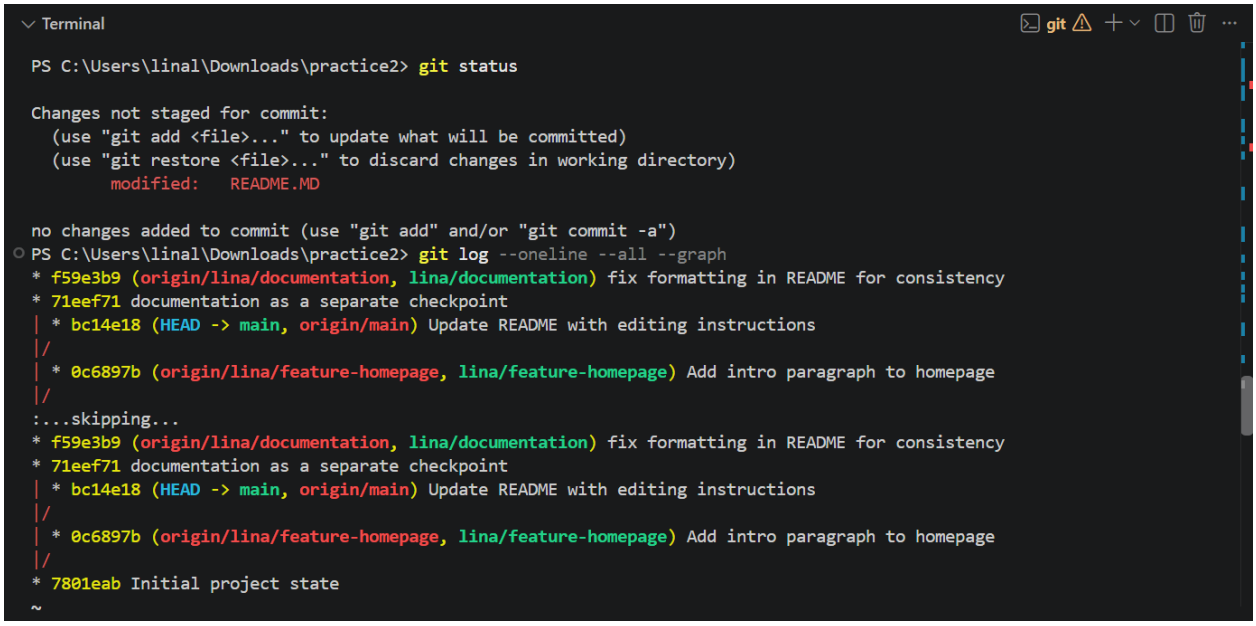
Objective

This lab practiced the core Git workflow for a small web project (index.html, style.css, README.md). It had two parts. Part 1 covered making separate commits, publishing to GitHub, and working on a feature branch. Part 2 covered syncing a change made on GitHub back to the local repository and inspecting changes before committing.

Part 1: Local commits, publishing and branching

I ended Part 1 with two commits published on main and one unmerged commit on a feature branch.

1. **Recorded the initial project state.** I staged all three files and committed them with `git add .` and `git commit -m "Initial project state"`.
2. **Added a Learning Goals section** to README.md under the title heading.
3. **Recorded the documentation change as its own checkpoint** with `git add README.md` and `git commit -m "Add Learning Goals section to README"`.
4. **Created an empty GitHub repository** named practice2, with no README, .gitignore or license.
5. **Connected the local repository** with `git remote add origin <repo URL>`, then checked the link with `git remote -v`.
6. **Published main** with `git branch -M main` and `git push -u origin main`.
7. **Created and switched to a feature branch** with `git switch -c feature-homepage`.
8. **Verified the current branch** with `git branch`. The asterisk marked feature-homepage.
9. **Added a short paragraph** inside the body of index.html.
10. **Recorded the homepage change on the feature branch** with `git add index.html` and `git commit -m "Add intro paragraph to homepage"`.
11. **Switched back to main** with `git switch main`. The new paragraph disappeared from index.html, since it exists only on feature-homepage.
12. **Left the feature branch unmerged**, as instructed.



Part 2: Syncing with the remote and inspecting changes

I pulled a commit made on GitHub into my local main, then prepared a new local change without committing it.

1. **Edited README.md on GitHub.** I opened the practice2 repository, edited one sentence with the pencil icon, and committed it directly to main as "Edit README on GitHub".
2. **Returned to the local repository** in VS Code and confirmed I was on main with `git branch`.
3. **Updated Git's knowledge of the remote** with `git fetch origin`. This downloaded the new commit without changing my working files.
4. **Checked whether I was behind** with `git status`. It reported that my branch was behind origin/main by 1 commit and could be fast-forwarded.
5. **Confirmed the remote commit in the history** with `git log --oneline --all --graph`. The GitHub commit appeared under origin/main, one step ahead of main. `git log --oneline main..origin/main` listed only that missing commit.
6. **Brought the remote change in** with `git pull`.
7. **Verified the README** with `cat README.md`. It contained the sentence edited on GitHub.
8. **Made one more local modification**, adding a line to README.md.
9. **Inspected the unstaged change** with `git diff`. Added lines were marked `+`.
10. **Staged the modification** with `git add README.md`.
11. **Inspected what is prepared for the next commit** with `git diff --staged`. Plain `git diff` then showed nothing, and `git status` listed README.md under "Changes to be committed".

Command summary

Command	What it does
`git add <file>` / `git add .`	Stages changes for the next commit
`git commit -m "msg"`	Records staged changes as a checkpoint
`git remote add origin <url>`	Links the local repo to GitHub
`git push -u origin main`	Publishes main and sets its upstream
`git switch -c <branch>`	Creates a branch and switches to it
`git switch <branch>`	Switches to an existing branch
`git branch`	Lists branches; `*` marks the current one
`git fetch origin`	Downloads remote commits without changing files
`git status`	Shows branch state and ahead/behind counts
`git log --oneline --all --graph`	Shows the commit history of all branches
`git pull`	Fetches and merges remote changes

Command	What it does
`git diff`	Shows unstaged changes
`git diff --staged`	Shows staged changes